

Horário escolar

Uma escola tem um conjunto de turmas (T), de disciplinas (D), de professores (P), com a respetiva disponibilidade, e de tipos de salas, comuns e especiais, em número limitado. O horário semanal tem 5 dias, cada um com 5 tempos. Cada disciplina d tem um professor $p(d)$ e uma carga semanal $c(d)$, um indicador de duplo período e um tipo de sala $s(d)$. Cada tipo de sala s tem Q_s salas disponíveis em simultâneo. Cada professor tem alguns tempos indisponíveis. Todos estes valores são lidos dos ficheiros CSV. Pretende-se gerar automaticamente um horário para cada turma.

Regras

- **R1.** Uma turma não pode ter duas aulas em simultâneo.
- **R2.** Cada disciplina cumpre exatamente a carga semanal exigida para cada turma.
- **R3.** No máximo, só pode haver uma aula da mesma disciplina por dia e por turma.
- **R4.** As disciplinas de duplo período têm de ser dadas em blocos de dois tempos consecutivos, no mesmo dia, e cada bloco contém exatamente uma aula.
- **R5.** Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas ou disciplinas diferentes.
- **R6.** Um professor só pode dar aulas nos tempos em que está disponível.
- **R7.** Cada aula ocupa uma sala. Disciplinas com `sala_especial` só podem usar salas desse tipo; as restantes usam salas comuns. O tempo o número de aulas a decorrer num tipo de sala pode exceder a quantidade desse tipo definida em `salas.csv`.

Objetivo

O número total de "buracos" no horário de cada professor deve ser minimizado. Um buraco é um tempo livre, no meio do dia, entre a primeira e a última aula desse professor nesse dia.

Construção incremental

Quando os recursos mudam ligeiramente, o horário deve ser reajustado sem recomeçar do zero e alterando o menor número de aulas.

Análise do problema

Este é um problema de alocação. Pretende-se alocar aulas (disciplinas de cada turma) aos tempos da semana, respeitando a disponibilidade dos professores e as salas existentes.

Existem T turmas, que podemos identificar por um índice $t \in [0..T-1]$, e D disciplinas, identificadas por $d \in [0..D-1]$. Cada aula é identificado por um par $(i, h) \in [0..4] \times [0..4]$, em que i é o dia e h o tempo desse dia.

Vamos usar uma família $x_{t,d,i,h}$ de variáveis binárias, com a seguinte semântica

$$x_{t,d,i,h} == 1 \quad \text{se e só se} \quad \text{a turma } t \text{ tiver a disciplina } d \text{ no dia } i, \text{ ao tempo } h.$$

Estas $T \times D \times 5 \times 5$ variáveis são convenientemente representadas numa matriz X instanciável com valores $\{0, 1\}$ $T \times D \times 5 \times 5$.

Estas $T \times D \times 5 \times 5$ variáveis são convenientemente representadas numa matriz X instanciável com valores $\{0, 1\}$.
costuma chamar *matriz de alocação*.

O professor e o tipo de sala de cada disciplina são dados do problema, e não variáveis de decisão. Por isso não fazem parte do objetivo e não aparecem nas restrições, através de $p(d)$ e $s(d)$.

Destaca-se ainda o seguinte:

Restrições

- 1. Uma turma não tem duas aulas em simultâneo.
- 2. Cada turma tem exatamente $c(d)$ tempos semanais de cada disciplina d .
- 3. Em cada dia, uma turma tem no máximo uma aula de cada disciplina sem duplo período, e no máximo um bloco de dois tempos de cada disciplina com duplo período.
- 4. As disciplinas de duplo período só podem ser dadas em blocos de dois tempos consecutivos, no mesmo dia.
- 5. Em cada tempo, um professor tem no máximo uma aula, somando todas as suas disciplinas e turmas.
- 6. Um professor não dá aulas nos tempos em que está indisponível.
- 7. Em cada tempo, o número de aulas num tipo de sala s não excede a quantidade de salas Q_s .

Implementação

Começamos por importar o CP-SAT do OR-TOOLS e criar o modelo 'modelo'.

Escolhemos o CP-SAT do OR-Tools porque o problema é naturalmente um problema de satisfação de restrições sobre variáveis booleanas. Não temos objetivo linear a minimizar: todas as regras se escrevem como somas de variáveis binárias, e o CP-SAT permite ainda sugerir dicas (AddHint), o que é útil na construção incremental. Os dados são lidos com pandas, que lê diretamente os CSV para tabelas e trata as linhas vazias (como sala_especial) sem código extra.

Depois obtemos os conjuntos T (turmas) e D (disciplinas) a partir dos dados lidos dos ficheiros CSV, e definimos a estrutura de dados para 5 tempos cada.

Em seguida, declaramos a matriz de alocação X como um dicionário de variáveis booleanas $x_{t,d,i,h}$.

Depois de resolver, o horário é guardado como uma **lista de tuplos** (t, d, i, h) , um por cada aula marcada. É neste formato que o código de validador, à contagem de buracos e à construção incremental.

Conjunto de dados dados

disciplina str	professor str	carga_semanal int64	duplo_periodo str	sala_especial str
Matemática	Prof. Ana	4	nao	
Português	Prof. Bruno	4	nao	
Ciências	Prof. Carla	3	nao	Laboratório
História	Prof. Diana	2	nao	

Inglês	Prof. Diana	2	nao	
Educação Física	Prof. Eduardo	2	sim	Ginásio
6 rows, 5 columns				
No selection				
Showing the first 10 of 15 rows. Increase the table's page_size to embed more in the static export.				
professor str	dia str	periodo int64		
Prof. Eduardo	Seg	1		
Prof. Eduardo	Seg	2		
Prof. Eduardo	Seg	3		
Prof. Eduardo	Ter	1		
Prof. Eduardo	Ter	2		
Prof. Eduardo	Ter	3		
Prof. Eduardo	Qua	1		
Prof. Eduardo	Qua	2		
Prof. Eduardo	Qua	3		
Prof. Eduardo	Qui	1		
15 rows, 3 columns				
No selection				
sala str	tipo str	quantidade int64		
Sala Normal	normal	6		
Laboratório	especial	1		
Ginásio	especial	1		
3 rows, 3 columns				
No selection				
turma str				
7ºA				
7ºB				
2 rows, 1 column				
No selection				

Passamos agora à modelação das restrições.

A restrição:

1. Uma turma não tem duas aulas em simultâneo

pode expressar-se da seguinte forma:

$$\forall t < T \cdot \forall i < I \cdot \forall h < H \cdot \sum_{d < D} x_{t,d,i,h} \leq 1$$

2. Cada turma tem exatamente $c(d)$ tempos semanais de cada disciplina d

pode expressar-se da seguinte forma:

$$\forall t < T \cdot \forall d < D \cdot \sum_{i < I} \sum_{h < H} x_{t,d,i,h} = c(d)$$

3. Em cada dia, uma turma tem no máximo uma aula de cada disciplina sem duplo período, e no máximo um bloco de dois de cada disciplina com duplo período. Divide-se em duas expressões, as com duplo período e as que não têm.

As sem duplo período pode expressar-se da seguinte forma:

$$\forall i < I \cdot \forall t < T \cdot \forall d < D, \text{duplo}(d) = \text{não} \quad \sum_{h < H} x_{t,d,i,h} \leq 1$$

As com duplo período pode expressar-se da seguinte forma (com $b_{t,d,i}$ binária):

$$\forall i < I \cdot \forall t < T \cdot \forall d < D, \text{duplo}(d) = \text{sim} \quad \sum_{h < H} x_{t,d,i,h} = 2 b_{t,d,i}$$

$$\forall i < I \cdot \forall t < T \cdot \forall d < D, \text{duplo}(d) = \text{sim} \cdot \forall h, k < H, k > h+1 \quad x_{t,d,i,h} + x_{t,d,i,k} \leq 1$$

4. Em cada tempo, um professor tem no máximo uma aula, somando todas as suas disciplinas e turmas

pode expressar-se da seguinte forma:

$$\forall p < P \cdot \forall i < I \cdot \forall h < H \cdot \sum_{t < T} \sum_{d < D, p(d)=p} x_{t,d,i,h} \leq 1$$

5. Um professor não dá aulas nos tempos em que está indisponível

pode expressar-se da seguinte forma:

$$\forall t < T \cdot \forall d < D \cdot \forall i < I \cdot \forall h < H, (p(d), i, h) \in \text{Ind} \quad x_{t,d,i,h} = 0$$

6. Em cada tempo, o número de aulas num tipo de sala s não excede a quantidade de salas Q_s

pode expressar-se da seguinte forma:

$$\forall s < S \cdot \forall i < I \cdot \forall h < H \cdot \sum_{t < T} \sum_{d < D, s(d)=s} x_{t,d,i,h} \leq Q_s$$

Validação do Horário

Quando o solver devolve um horário, este cumpre sempre o modelo que lhe demos, mas isso não garante que o modelo este, a restrição esquecida ou mal formulada passa despercebida. Por isso escrevemos um **validador independente**, `verificar(pasta)` recebe o nome da pasta dos dados e um horário já gerado (lista de tuplos (t, d, i, h)) e confirma, regra a regra, que o horário

O validador não usa o modelo CP-SAT, as suas variáveis nem `lista_de_dados`: relê os dados dos ficheiros CSV e recalcula tu horário. Devolve a lista de violações encontradas, e uma lista vazia significa que o horário é válido.

Testes do validador

Um validador que nunca falha não prova nada. Por isso, para cada requisito, partimos de um horário válido, introduzimos de violação desse requisito e confirmamos que `verificar` a deteta. Confirmamos também que o horário gerado pelo solver não violação.

Dados diferentes

Para mostrar que nada está escrito diretamente no código, repetimos o fluxo completo com um conjunto de dados diferente, com uma turma e uma disciplina a mais e outra exceção de disponibilidade. Só muda o nome da pasta lida.

[] 0 Items

9 testes passaram

Objetivo: minimizar os buracos dos professores

Um **buraco** é um tempo livre de um professor, no meio do dia, com pelo menos uma aula antes e uma aula depois. Os tempos fim do dia não contam.

Para os contar dentro do modelo, introduzimos três famílias de variáveis binárias, para cada professor p , dia i e tempo h :

- $a_{p,i,h}$ vale 1 se o professor p tem aula no tempo h no dia i ;
- $m_{p,i,h}$ vale 1 se o professor p tem uma aula num tempo anterior a h , nesse dia;
- $n_{p,i,h}$ vale 1 se o professor p tem uma aula num tempo posterior a h , nesse dia.

O valor de a obtém-se somando as aulas de todas as disciplinas do professor, em todas as turmas:

$$a_{p,i,h} = \sum_{t < T} \sum_{d < D, p(d)=p} x_{t,d,i,h}$$

As variáveis m e n ficam limitadas inferiormente pelos tempos em causa:

$$\forall k < h \cdot m_{p,i,h} \geq a_{p,i,k} \qquad \forall k > h \cdot n_{p,i,h} \geq a_{p,i,k}$$

Basta o limite inferior: se houver aula antes, m é forçada a 1, e o solver nunca ganha em pô-la a 1 sem necessidade, porque isso cria os buracos.

Introduzimos ainda uma variável binária $b_{p,i,h}$ por tempo, que indica um buraco. Há buraco quando existe aula antes, existe aula depois, ou não existe aula no próprio tempo:

$$b_{p,i,h} \geq m_{p,i,h} + n_{p,i,h} - a_{p,i,h} - 1$$

Quando as três condições se verificam, o lado direito vale 1 e b é forçada a 1. Nos outros casos vale 0 ou menos, e b pode ser 0. O primeiro e o último tempo do dia nunca podem ser buracos, só criamos buracos nos tempos interiores.

O objetivo é minimizar o número total de buracos:

$$\min \sum_{p < P} \sum_{i < I} \sum_{h < H} b_{p,i,h}$$

Esta função objetivo não altera as restrições: apenas escolhe, entre os horários válidos, os que têm menos buracos. Por isso o modelo sempre vai passar no validador.

Verificação independente. Para confirmar o resultado sem usar o modelo, a função `contar_buracos` percorre o horário já gerado, para cada professor e dia, calcula o intervalo entre a primeira e a última aula menos o número de aulas. Depois comparamos este valor com o valor retornado por `solver.ObjectiveValue()`: se a solução for `OPTIMAL`, têm de ser iguais (somando o custo das preferências, quando existem).

```
restricao_sem_aulas_simultaneo 50
restricao_sem_duplo_periodo 50
restricao_com_duplo_periodo 70
```

```
restricao_com_dueto_periudo 70
restricao_professor_maximo_uma_aula_por_tempo 125
restricao_professor_indisponivel 30
restricao_salas 75
restricao_carga_semanal 12
```

```
<class 'list'> 34
```

Estado: OPTIMAL

Objetivo do solver: 0.0

Buracos (independente): 0 | preferências: 0 | total: 0

Sem objetivo: 2 buracos

Com objetivo: 0 buracos

Horário válido: True

7ºA

1º tempo: Português	-	-	Ciências	Português
2º tempo: Ciências	-	-	Inglês	Ciências
3º tempo: Matemática	-	Inglês	Português	Matemática
4º tempo: Educação Física	Matemática	História	História	-
5º tempo: Educação Física	Português	Matemática	-	-

7ºB

1º tempo: Ciências	História	-	-	Ciências
2º tempo: Inglês	Ciências	-	Matemática	Português
3º tempo: História	-	Português	Inglês	-
4º tempo: Matemática	Português	Educação Física	Português	Matemática
5º tempo: -	Matemática	Educação Física	-	-

Construção Incremental

Quando os recursos mudam ligeiramente, resolver o problema do zero tem dois problemas: é mais lento e, sobretudo, o solver encontra qualquer entre os que cumprem as regras, que costuma ser muito diferente do anterior. Para a escola isso significa mudar aulas que precisavam de mudar.

A nossa abordagem usa o horário anterior H_0 (lista de aulas) de duas formas:

1. **Ponto de partida.** Sugerimos ao solver os valores de H_0 (AddHint), para começar a procura perto de uma solução conhecida.
2. **Custo por mudança.** Acrescentamos ao objetivo um termo que penaliza cada aula de H_0 que deixa de estar em H_1 :

$$\min \text{ peso} \cdot \sum_{(t,d,i,h) \in H_0} (1 - x_{t,d,i,h}^1) + \sum_{p,i,h} b_{p,i,h}$$

O peso (100) é muito maior do que o número de buracos que se pode ganhar, por isso a estabilidade tem prioridade sobre o número de aulas alteradas. Escolha: aceitamos mais buracos se isso evitar mover uma aula.

O H_1 cumpre as mesmas regras R1–R7, porque usa o mesmo `construir_modelo` com os dados novos. As aulas de H_0 que não estão no novo modelo (por exemplo, de uma turma que desapareceu) são ignoradas. O número de aulas alteradas é o número de aulas de H_0 que não estão em H_1 .

H1.

Resultados com dados → dados_v2 (tempo médio de 5 execuções, só da resolução):

	Tempo	Aulas alteradas	Válido
Do zero	0,054 s	25	sim
Incremental	0,038 s	2	sim

O ganho principal é na estabilidade: o horário novo mantém quase tudo e muda 2 aulas, em vez de 25. A diferença de tempo é pequena, porque a instância é pequena e ambos os métodos resolvem em milissegundos. O benefício de velocidade só deve ser visto em instâncias maiores.

Outros cenários

Para mostrar que a abordagem não depende do cenário de dados_v2, aplicámo-la a mais alterações de recursos, todas feitas sem mexer no código. Os dados alterados são gravados numa pasta própria (guardar_dados), para que o validador os possa usar em outros casos.

Alteração	Aulas alteradas (incremental)	Aulas alteradas (do zero)
Disponibilidade (dados_v2)	2	25
Turma e disciplina novas	3	27
Professor substituído	0	30
Sala Normal com menos salas	6	33

Em todos os casos os dois horários são válidos, e o incremental muda muito menos aulas. A avaria de uma sala modela-se bem com o tipo em salas.csv, porque o formato dos dados só guarda a quantidade por tipo de sala e não a disponibilidade por tempo.

```
{ 3 Items
  "violações H0": [] 0 Items
  "violações H1": [] 0 Items
  "aulas alteradas": 1
}
```

Do zero: 0.024 s, 30 aulas alteradas
Incremental: 0.014 s, 1 aulas alteradas
Válidos: True True

Professor substituído | incremental: 0 | do zero: 25 | válidos: True True
Sala Normal reduzida | incremental: 7 | do zero: 30 | válidos: True True

Incremental: 2 aulas alteradas
Do zero: 28 aulas alteradas
Válidos: True True

Turma 7ºA

Index0str	Segstr	Terstr	Quastr	Quistr	Sestr
1º tempo	Português	-	-	Ciências	P
2º tempo	Ciências	-	-	Inglês	C
3º tempo	Matemática	-	Inglês	Português	M
4º tempo	Educação Física	Matemática	História	História	-
5º tempo	Educação Física	Português	Matemática	-	-

5 rows, 5 columns
No selection

Turma 7ºB

Index0str	Segstr	Terstr	Quastr	Quistr	Sestr
1º tempo	Ciências	História	-	-	C
2º tempo	Inglês	Ciências	-	Matemática	P
3º tempo	História	-	Português	Inglês	-
4º tempo	Matemática	Português	Educação Física	Português	M
5º tempo	-	Matemática	Educação Física	-	-

5 rows, 5 columns
No selection

7ºA, H0

Index0str	Segstr	Terstr	Quastr	Quistr	Sexstr
1º tempo	Português	-	-	Ciências	Português
2º tempo	Ciências	-	-	Inglês	Ciências
3º tempo	Matemática	-	Inglês	Português	Matemática
4º tempo	Educação Física	Matemática	História	História	-
5º tempo	Educação Física	Português	Matemática	-	-

5 rows, 5 columns
No selection

7ºA, H1

Index0str	Segstr	Terstr	Quastr
1º tempo	Português	-	-
2º tempo	Ciências	-	-
3º tempo	Matemática	-	Inglês
4º tempo	Educação Física	Matemática	História
5º tempo	Educação Física	Português	Matemática

5 rows, 5 columns
No selection

Escala (bónus)

Para estudar o comportamento com mais turmas, geramos conjuntos de dados sintéticos (gerar_dados_escala) com 16 discos
semanal 1, cada uma com um professor próprio, e 4 a 24 turmas. Os dados continuam a ser lidos de ficheiros CSV, sem altera

Turmas	Variáveis	Tempo de resolução (s)	Resultado
4	1 600	0,22	válido
8	3 200	0,35	válido
12	4 800	0,48	válido
16	6 400	0,79	válido
20	8 000	0,86	válido
24	9 600	1,05	válido

Em todos os casos o horário cumpre as regras (verificado pelo validador independente) e o tempo cresce de forma aproximadamente linear com o número de variáveis, ficando perto de 1 s para 9 600 variáveis.

Limites encontrados. Até às 24 turmas não encontrámos um limite computacional, mas encontrámos um limite estrutural dos dados. No formato atual, cada turma tem todas as disciplinas e cada disciplina um único professor, pelo que a carga de um professor cresce com o número de turmas. Com 8 disciplinas de carga 2, um professor dá $2N$ tempos por semana e só tem 25, logo o problema fica impossível para $N > 12$ (confirmámos com 16: o solver prova rapidamente que não há solução). Por isso os dados de escala usam carga 1 e mais disciplinas, até 25 turmas.

A dificuldade de uma instância depende não só do tamanho mas de quão apertada está: com salas, professores ou tempos mais restritos, tem de explorar muito mais, e é aí que o tempo pode crescer depressa.

turmas	variáveis	tempo (s)	resultado	
int64	int64	float64	str	
4	1600	0,11	válido	
8	3200	0,18	válido	
12	4800	0,26	válido	
16	6400	0,72	válido	
20	8000	0,42	válido	
24	9600	0,51	válido	
6 rows, 4 columns				
No selection				

Preferências dos professores (bónus)

Além dos buracos, os professores podem indicar tempos em que preferem não dar aulas. Estas preferências são lidas de `preferencias.txt` (professor, dia, período e penalização), um ficheiro opcional: se não existir, o modelo comporta-se como antes.

Uma preferência não é uma restrição, é um custo: dar uma aula num tempo penalizado soma a penalização ao objetivo. O objetivo é

$$\min \sum_{p,i,h} b_{p,i,h} + \sum_{p,i,h} \text{pen}_{p,i,h} \cdot a_{p,i,h}$$

em que $\text{pen}_{p,i,h}$ é a penalização lida do ficheiro (0 se não houver). Uma penalização de 1 vale tanto como um buraco, e vale o mesmo peso à preferência. Como é um custo e não uma regra, o validador não a verifica: o horário pode, em princípio, usar um tempo proibido.

peso a preferência. Como é um custo e não uma regra, o validador não a verifica. O horário pode, em princípio, usar um tempo para compensar noutro critério. O custo é recalculado de forma independente por `custo_preferencias(pasta, horario)`.

Resultado. Com o Prof. Bruno a preferir não dar aulas aos dois primeiros tempos de cada dia (penalização 3), o horário gerado com as preferências tem um custo de 9 (três aulas nesses tempos), e com elas o custo desce para 0. O número de buracos mantém-se o mesmo e continua válido. Os dados deste exemplo são gravados em `dados_preferencias`, para que o custo seja avaliado a partir dos dados do validador.

```
Custo das preferências sem elas: 9
Custo das preferências com elas: 0
Buracos sem elas: 0
Buracos com elas: 0
Válido: True
```

Uso de Ferramenta LLM

Durante a realização do trabalho foi utilizado o Claude como ferramenta de apoio à implementação, depuração e revisão do código.

O uso da ferramenta ocorreu principalmente nas seguintes partes:

- revisão da função `menos_buracos`, onde faltava a restrição que liga a variável de buraco b às variáveis a , m e n , e a sua implementação;
- discussão sobre a independência do validador e da contagem de buracos em relação ao modelo, levando a que `validar_horario` e `custo_preferencias` passassem a reler os ficheiros CSV a partir do nome da pasta;
- uniformização do formato do horário como lista de tuplos (t, d, i, h) , com a função `extrair_horario`, e adaptação das funções para esse formato;
- definição da validação cruzada entre o valor do objetivo devolvido pelo solver e o cálculo independente dos buracos e do custo;
- adaptação da construção incremental ao novo formato do horário e criação da função `guardar_dados` para validar os exemplos a partir de ficheiros;
- resolução de erros de execução no Marimo, como variáveis não definidas entre células, células com funções aninhadas e problemas com a versão instalada do OR-Tools (`StatusName`);
- fixação dos parâmetros do solver (`num_workers` e `random_seed`) para tornar os resultados reprodutíveis;
- diagnóstico do exemplo das preferências, que inicialmente não mostrava efeito porque a professora escolhida nunca tinha aulas penalizados, e definição de um exemplo com efeito visível.

O código foi desenvolvido de forma incremental: era apresentada uma tentativa de implementação ou uma mensagem de erro, analisava-se o problema e a respetiva correção. As sugestões foram testadas no notebook antes de serem mantidas, e os resultados apresentados no relatório foram obtidos pela execução do próprio notebook.

O diálogo utilizado durante a resolução encontra-se disponível em: <https://claude.ai/share/af2f7552-ebd7-4024-ad27-f44471>